Declaration

```
void atomic_flag_clear_explicit(
    volatile atomic_flag* flag, memory_order order) noexcept;
void atomic_flag_clear_explicit(
    atomic_flag* flag, memory_order order) noexcept;
```

Effects

```
return flag->clear(order);
```

D.3.8. std::atomic class template

The `std::atomic` class provides a wrapper with atomic operations for any type that satisfies the following requirements.

The template parameter `BaseType` must

- Have a trivial default constructor
- Have a trivial copy-assignment operator
- Have a trivial destructor
- Be bitwise-equality comparable

This basically means that `std::atomic<`*some-built-in-type*`>` is fine, as is `std::atomic<`*some-simple-struct*`>`, but things like `std::atomic<std::string>` are not.

In addition to the primary template, there are specializations for the built-in integral types and pointers to provide additional operations such as `x++`.

Instances of `std::atomic` are not `CopyConstructible` or `CopyAssignable`, because these operations can't be performed as a single atomic operation.

Class definition

```
template<typename BaseType>
struct atomic
```